

UIDE LÓGICA DE PROGRAMACIÓN

NOMBRE:

Tripuliche Ronquillo Lizbeth
Alvarado Ruiz Janneli

DESCRIPCIÓN DE LA ASIGNACIÓN:

Desafío: MiniTienda – registro y análisis de ventas

Implementar un programa de consola que:

1. Mantiene un catálogo (tuplas), precios/stock (diccionarios)
2. Registra ventas (listas + pandas DataFrame)
3. Guarda/lee datos desde un CSV (archivos)
4. Calcula métricas con NumPy
5. Grafica ingresos por producto con Matplotlib
6. Tiene un menú con bucle while y control de flujo completo.

ENLACE DE GOOGLE COLAB:

https://colab.research.google.com/drive/1Mhw95zYP2g_1mJWkBuf88YbSS11lhsWO?usp=sharing

EXPLICACIÓN BREVE DE LO REALIZADO:

El programa desarrollado consiste en un sistema de gestión y análisis de ventas en consola implementado en Python con una estructura modular, utilizando estructuras de control como ciclos while y for, condicionales if, elif y else, e instrucciones break y continue para manejar el flujo y la ejecución del menú. Para la administración de datos, el sistema emplea tuplas para almacenar el catálogo de productos (Laptop, Mouse, Teclado, Monitor y Audífonos), diccionarios para gestionar los precios y el stock disponible, y listas para registrar las ventas realizadas.

Durante el proceso de registro, se realizan validaciones estrictas de existencia de producto, stock suficiente y cantidades mayores a cero, aplicando un descuento automático del 5 % cuando la compra es igual o superior a 10 unidades. El procesamiento analítico se apoya en la biblioteca Pandas para convertir las ventas en un DataFrame, agrupar la información por producto mediante groupby y gestionar la exportación y lectura del archivo ventas.csv, el cual contiene 12 ventas de prueba.

Por su parte, la biblioteca NumPy procesa los arreglos numéricos de ingresos y unidades para calcular el total, promedio y desviación estándar, así como el ingreso promedio por unidad previniendo divisiones por cero. Para la representación gráfica, Matplotlib genera un diagrama de barras con los ingresos por producto y lo exporta en formato PNG como ingresos.png.

El control de errores y eventos se realiza mediante bloques try, except, else y finally, registrando operaciones exitosas y fallos como el intento de venta de productos inexistentes o falta de stock en el archivo log.txt. Finalmente, el sistema cumple con cuatro retos adicionales que incluyen la incorporación de nuevos productos actualizando su precio y stock (Reto A), la exportación de la gráfica a PNG (Reto B), la aplicación automática de descuentos por volumen (Reto C) y el registro detallado de intentos de ventas no válidas en el archivo de logs (Reto D).

Respuestas a las preguntas de la actividad

¿Qué parte la hizo Pandas?

Pandas se utilizó para transformar las ventas almacenadas en las estructuras del programa en un DataFrame, organizar los registros, agruparlos mediante groupby para obtener un resumen de unidades e ingresos por producto, guardar los datos en ventas.csv y leer nuevamente la información almacenada en dicho archivo.

¿Qué parte la hizo NumPy?

NumPy se utilizó para realizar los cálculos numéricos sobre los arreglos de ingresos y unidades vendidas. Con esta biblioteca se obtuvieron la suma (sum), el promedio (mean) y la desviación estándar (std), además del cálculo del ingreso promedio por unidad.

¿Dónde se utilizó try/except y por qué?

try/except se utilizó en diferentes partes del programa para controlar posibles errores. Se empleó al leer el archivo CSV para controlar el caso de que ventas.csv no exista, y también durante el ingreso de datos para manejar valores inválidos. Además, se utiliza para controlar errores inesperados durante la ejecución. Esto permite que el programa muestre un mensaje adecuado en lugar de detenerse completamente cuando ocurre un problema. También se utilizaron else y finally como parte del manejo de excepciones.

¿Qué estructuras son tuplas, listas y diccionarios en el código?

Las tuplas se utilizan principalmente para construir el catálogo de productos, almacenando el ID y nombre de cada producto. Las listas se utilizan para almacenar temporalmente las ventas registradas y los identificadores de las ventas. Los diccionarios se utilizan para almacenar los precios y el stock de cada producto, y también para representar la información completa de cada venta antes de incorporar al conjunto de datos utilizado por Pandas.

ENTRADAS:

```
import os
from datetime import datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

ARCHIVO_CSV = "ventas.csv"
ARCHIVO_LOG = "log.txt"
ARCHIVO_GRAFICO = "ingresos.png"

catalogo = (
    (1, "Laptop"),
    (2, "Mouse"),
    (3, "Teclado"),
    (4, "Monitor"),
    (5, "Audifonos")
)

precios = {
    1: 800.00,
    2: 20.00,
    3: 35.00,
    4: 180.00,
    5: 45.00
}

stock = {
    1: 10,
    2: 50,
    3: 30,
    4: 15,
    5: 25
}
```

```
ventas = []
ids_ventas = []

def escribir_log(mensaje):
    try:
        with open(ARCHIVO_LOG, "a", encoding="utf-8") as archivo:
            fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            archivo.write(f"[{fecha}] {mensaje}\n")
    except Exception as error:
        print(f"Error al escribir en el archivo de registro: {error}")

def obtener_nombre(producto_id):
    for pid, nombre in catalogo:
        if pid == producto_id:
            return nombre
    return None

def siguiente_id_venta():
    if len(ids_ventas) == 0:
        return 1
    return max(ids_ventas) + 1

def mostrar_catalogo():
    print("\nCatálogo de productos")
    print(f'{"ID":<6}{"Producto":<20}{"Precio":<15}{"Stock":<10}')
    print("-" * 55)

    for producto_id, nombre in catalogo:
        print(
            f"{producto_id:<6}"
            f"{nombre:<20}"
```

```
f"{precios[producto_id]:<14.2f}"
f"{stock[producto_id]:<10}"
)

def registrar_venta(producto_id, unidades):
    if producto_id not in precios:
        print("Error: el producto no existe en el catálogo.")
        escribir_log(
            f"INTENTO FALLIDO - Producto ID {producto_id} no existe en el catálogo."
        )
        return False

    if unidades <= 0:
        print("Error: las unidades deben ser mayores que cero.")
        escribir_log(
            f"VENTA RECHAZADA - Cantidad inválida: {unidades}"
        )
        return False

    if unidades > stock[producto_id]:
        print(
            f"Error: stock insuficiente. Stock disponible: "
            f"{stock[producto_id]}"
        )
        escribir_log(
            f"VENTA RECHAZADA - Stock insuficiente para producto {producto_id}."
        )
        return False

    nombre = obtener_nombre(producto_id)
    precio_unitario = precios[producto_id]

    if unidades >= 10:
        porcentaje_descuento = 0.05
```

```
else:
    porcentaje_descuento = 0

subtotal = precio_unitario * unidades
valor_descuento = subtotal * porcentaje_descuento
total = subtotal - valor_descuento

venta_id = siguiente_id_venta()

venta = {
    "venta_id": venta_id,
    "producto_id": producto_id,
    "producto": nombre,
    "unidades": unidades,
    "precio_unitario": precio_unitario,
    "subtotal": subtotal,
    "descuento": valor_descuento,
    "total": total,
    "fecha": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
}

ventas.append(venta)
ids_ventas.append(venta_id)
stock[producto_id] -= unidades

escribir_log(
    f"VENTA REGISTRADA - ID {venta_id} - {nombre} - "
    f"Unidades: {unidades} - Total: ${total:.2f}"
)

print("\nVenta registrada correctamente.")
print(f"ID de venta: {venta_id}")
print(f"Producto: {nombre}")
print(f"Unidades: {unidades}")
print(f"Precio unitario: ${precio_unitario:.2f}")
```

```

    print(f"Subtotal: ${subtotal:.2f}")
    print(f"Descuento: ${valor_descuento:.2f}")
    print(f"Total: ${total:.2f}")

    return True

def obtener_dataframe():
    if len(ventas) == 0:
        return pd.DataFrame()
    return pd.DataFrame(ventas)

def guardar_csv():
    if len(ventas) == 0:
        print("No existen ventas para guardar.")
        return False

    df = pd.DataFrame(ventas)
    df.to_csv(ARCHIVO_CSV, index=False, encoding="utf-8")

    print(f"Ventas guardadas en {ARCHIVO_CSV}.")
    return True

def cargar_csv():
    try:
        df = pd.read_csv(ARCHIVO_CSV)

    except FileNotFoundError:
        print(f"Error: el archivo {ARCHIVO_CSV} no existe.")
        escribir_log(
            f"LECTURA FALLIDA - No existe {ARCHIVO_CSV}."
        )
    return pd.DataFrame()

```

```

except Exception as error:
    print(f"Error al leer el archivo: {error}")
    escribir_log(f"ERROR DE LECTURA - {error}")
    return pd.DataFrame()

else:
    print(f"{ARCHIVO_CSV} leído correctamente.")
    return df

finally:
    print("Proceso de lectura del CSV finalizado.")

def mostrar_resumen_pandas():
    df = obtener_dataframe()

    if df.empty:
        print("No existen ventas registradas.")
        return

    print("\nVentas registradas:")
    print(df.to_string(index=False))

    resumen = (
        df.groupby("producto", as_index=False)
        .agg(
            unidades_vendidas=("unidades", "sum"),
            ingresos=("total", "sum")
        )
        .sort_values("ingresos", ascending=False)
    )

    print("\nResumen por producto:")
    print(resumen.to_string(index=False))

```

```

    )

    print("\nResumen por producto:")
    print(resumen.to_string(index=False))

    guardar_csv()

def calcular_metricas():
    df = obtener_dataframe()

    if df.empty:
        print("No existen ventas para calcular métricas.")
        return

    ingresos = np.array(df["total"], dtype=float)
    unidades = np.array(df["unidades"], dtype=float)

    total_ingresos = np.sum(ingresos)
    promedio_ingresos = np.mean(ingresos)
    desviacion_ingresos = np.std(ingresos)
    total_unidades = np.sum(unidades)
    promedio_unidades = np.mean(unidades)

    print("\nMétricas:")
    print(f"Suma de ingresos: ${total_ingresos:.2f}")
    print(f"Media de ingresos por venta: ${promedio_ingresos:.2f}")
    print(f"Desviación estándar: ${desviacion_ingresos:.2f}")
    print(f"Suma de unidades vendidas: {total_unidades:.0f}")
    print(f"Media de unidades por venta: {promedio_unidades:.2f}")

    if total_unidades == 0:
        print("No se puede calcular el ingreso promedio por unidad.")

```

```

    else:
        ingreso_por_unidad = total_ingresos / total_unidades
        print(f"Ingreso promedio por unidad: ${ingreso_por_unidad:.2f}")

def graficar_ingresos():
    df = obtener_dataframe()

    if df.empty:
        print("No existen ventas para generar la gráfica.")
        return False

    resumen = (
        df.groupby("producto")["total"]
        .sum()
        .sort_values(ascending=False)
    )

    plt.figure(figsize=(9, 5))
    resumen.plot(kind="bar")
    plt.title("Ingresos por producto")
    plt.xlabel("Producto")
    plt.ylabel("Ingresos ($)")
    plt.xticks(rotation=30, ha="right")
    plt.tight_layout()
    plt.savefig(ARCHIVO_GRAFICO, dpi=150, bbox_inches="tight")
    plt.show()
    plt.close()

    print(f"Gráfico guardado como {ARCHIVO_GRAFICO}.")
    return True

def agregar_producto():
    global catalogo

```

```

try:
    nuevo_id = int(input("Ingrese el ID del nuevo producto: "))

    if nuevo_id in precios:
        print("Error: ese ID ya existe.")
        return

    nombre = input("Ingrese el nombre del producto: ").strip()

    if nombre == "":
        print("Error: el nombre no puede estar vacío.")
        return

    nuevo_precio = float(input("Ingrese el precio: "))
    nuevo_stock = int(input("Ingrese el stock inicial: "))

    if nuevo_precio < 0 or nuevo_stock < 0:
        print("Error: precio y stock no pueden ser negativos.")
        return

    catalogo = catalogo + ((nuevo_id, nombre),)
    precios[nuevo_id] = nuevo_precio
    stock[nuevo_id] = nuevo_stock

    escribir_log(
        f"PRODUCTO AGREGADO - ID {nuevo_id} - {nombre} - "
        f"Precio ${nuevo_precio:.2f} - Stock {nuevo_stock}"
    )

    print("Producto agregado correctamente.")

except ValueError:
    print("Error: ingrese valores numéricos válidos.")

except Exception as error:

```

```

print(f"Ocurrió un error: {error}")

finally:
    print("Proceso de agregar producto finalizado.")

def generar_ventas_prueba():
    ventas.clear()
    ids_ventas.clear()

    stock[1] = 10
    stock[2] = 50
    stock[3] = 30
    stock[4] = 15
    stock[5] = 25

    ventas_prueba = [
        (1, 1),
        (2, 3),
        (3, 2),
        (4, 1),
        (5, 4),
        (2, 10),
        (3, 5),
        (4, 2),
        (5, 3),
        (2, 6),
        (3, 10),
        (5, 2)
    ]

    for producto_id, unidades in ventas_prueba:
        registrar_venta(producto_id, unidades)

    guardar_csv()

```

```

def pruebas():
    print("\nPrueba de producto inexistente:")
    registrar_venta(999, 1)

    print("\nPrueba de cantidad inválida:")
    registrar_venta(1, 0)

    print("\nPrueba de stock insuficiente:")
    registrar_venta(1, 999)

    print("\nLectura del archivo CSV:")
    df = cargar_csv()

    if not df.empty:
        print(df.to_string(index=False))

def mostrar_menu():
    print("\nsistema de ventas")
    print("1) Mostrar catálogo")
    print("2) Registrar venta")
    print("3) Mostrar ventas y resumen Pandas")
    print("4) Calcular métricas con Numpy")
    print("5) Graficar ingresos por producto")
    print("6) Exportar gráfico a PNG")
    print("7) Agregar producto")
    print("8) Guardar ventas en CSV")
    print("9) Leer ventas desde CSV")
    print("0) Salir")

def menu():
    while True:
        mostrar_menu()
        opcion = input("Seleccione una opción: ").strip()

```

```

try:
    if opcion == "1":
        mostrar_catologo()

    elif opcion == "2":
        try:
            producto_id = int(input("Ingrese el ID del producto: "))
            unidades = int(input("Ingrese la cantidad: "))
            registrar_venta(producto_id, unidades)

        except ValueError:
            print("Error: ingrese números enteros válidos.")
            escribir_log(
                "INPUT INVÁLIDO - Registro de venta."
            )

    elif opcion == "3":
        mostrar_resumen_pandas()

    elif opcion == "4":
        calcular_metricas()

    elif opcion == "5":
        graficar_ingresos()

    elif opcion == "6":
        if graficar_ingresos():
            print("Gráfico exportado correctamente.")
        else:
            continue

    elif opcion == "7":
        agregar_producto()

    elif opcion == "8":
        guardar_csv()

```

```

df = cargar_csv()

if not df.empty:
    print(df.to_string(index=False))

elif opcion == "0":
    print("Programa finalizado.")
    break

else:
    print("Opción inválida.")
    escribir_log(
        f"OPCIÓN INVÁLIDA - Usuario ingresó: {opcion}"
    )
    continue

except Exception as error:
    print(f"Error inesperado: {error}")
    escribir_log(f"ERROR INESPERADO - {error}")

finally:
    print("Fin de la operación.")

generar_ventas_prueba()
pruebas()
calcular_metricas()
graficar_ingresos()

print("\nArchivos generados:")
for archivo in [ARCHIVO_CSV, ARCHIVO_LOG, ARCHIVO_GRAFICO]:
    if os.path.exists(archivo):
        print(f"{archivo}: creado correctamente")
    else:
        print(f"{archivo}: no encontrado")

```

SALIDAS:

<pre> Venta registrada correctamente. ID de venta: 1 ... Producto: Laptop Unidades: 1 Precio unitario: \$800.00 Subtotal: \$800.00 Descuento: \$0.00 Total: \$800.00 Venta registrada correctamente. ID de venta: 2 Producto: Mouse Unidades: 3 Precio unitario: \$20.00 Subtotal: \$60.00 Descuento: \$0.00 Total: \$60.00 Venta registrada correctamente. ID de venta: 3 Producto: Teclado Unidades: 2 Precio unitario: \$35.00 Subtotal: \$70.00 Descuento: \$0.00 Total: \$70.00 Venta registrada correctamente. ID de venta: 4 Producto: Monitor Unidades: 1 Precio unitario: \$180.00 Subtotal: \$180.00 Descuento: \$0.00 Total: \$180.00 </pre>	<pre> Venta registrada correctamente. ID de venta: 5 ... Producto: Audifonos Unidades: 4 Precio unitario: \$45.00 Subtotal: \$180.00 Descuento: \$0.00 Total: \$180.00 Venta registrada correctamente. ID de venta: 6 Producto: Mouse Unidades: 10 Precio unitario: \$20.00 Subtotal: \$200.00 Descuento: \$10.00 Total: \$190.00 Venta registrada correctamente. ID de venta: 7 Producto: Teclado Unidades: 5 Precio unitario: \$35.00 Subtotal: \$175.00 Descuento: \$0.00 Total: \$175.00 Venta registrada correctamente. ID de venta: 8 Producto: Monitor Unidades: 2 Precio unitario: \$180.00 Subtotal: \$360.00 Descuento: \$0.00 Total: \$360.00 </pre>
--	--

```

Venta registrada correctamente.
ID de venta: 9
*** Producto: Audifonos
Unidades: 3
Precio unitario: $45.00
Subtotal: $135.00
Descuento: $0.00
Total: $135.00

Venta registrada correctamente.
ID de venta: 10
Producto: Mouse
Unidades: 6
Precio unitario: $20.00
Subtotal: $120.00
Descuento: $0.00
Total: $120.00

Venta registrada correctamente.
ID de venta: 11
Producto: Teclado
Unidades: 10
Precio unitario: $35.00
Subtotal: $350.00
Descuento: $17.50
Total: $332.50

Venta registrada correctamente.
ID de venta: 12
Producto: Audifonos
Unidades: 2
Precio unitario: $45.00
Subtotal: $90.00
Descuento: $0.00
Total: $90.00
Ventas guardadas en ventas.csv.

Prueba de producto inexistente:
Error: el producto no existe en el catálogo.

```

```

Prueba de cantidad inválida:
*** Error: las unidades deben ser mayores que cero.

Prueba de stock insuficiente:
Error: stock insuficiente. Stock disponible: 9

Lectura del archivo CSV:
ventas.csv leído correctamente.
Proceso de lectura del CSV finalizado.

```

venta_id	producto_id	producto	unidades	precio_unitario	subtotal	descuento	total	fecha
1	1	Laptop	1	800.0	800.0	0.0	800.0	2026-08-17 16:51:08
2	2	Mouse	3	20.0	60.0	0.0	60.0	2026-08-17 16:51:08
3	3	Teclado	2	35.0	70.0	0.0	70.0	2026-08-17 16:51:08
4	4	Monitor	1	180.0	180.0	0.0	180.0	2026-08-17 16:51:08
5	5	Audifonos	4	45.0	180.0	0.0	180.0	2026-08-17 16:51:08
6	2	Mouse	10	20.0	200.0	10.0	190.0	2026-08-17 16:51:08
7	3	Teclado	5	35.0	175.0	0.0	175.0	2026-08-17 16:51:08
8	4	Monitor	2	180.0	360.0	0.0	360.0	2026-08-17 16:51:08
9	5	Audifonos	3	45.0	135.0	0.0	135.0	2026-08-17 16:51:08
10	2	Mouse	6	20.0	120.0	0.0	120.0	2026-08-17 16:51:08
11	3	Teclado	10	35.0	350.0	17.5	332.5	2026-08-17 16:51:08
12	5	Audifonos	2	45.0	90.0	0.0	90.0	2026-08-17 16:51:08

```

Métricas:
Suma de ingresos: $2692.50
Media de ingresos por venta: $224.38
Desviación estándar: $195.10
Suma de unidades vendidas: 49
Media de unidades por venta: 4.08
Ingreso promedio por unidad: $54.95

```

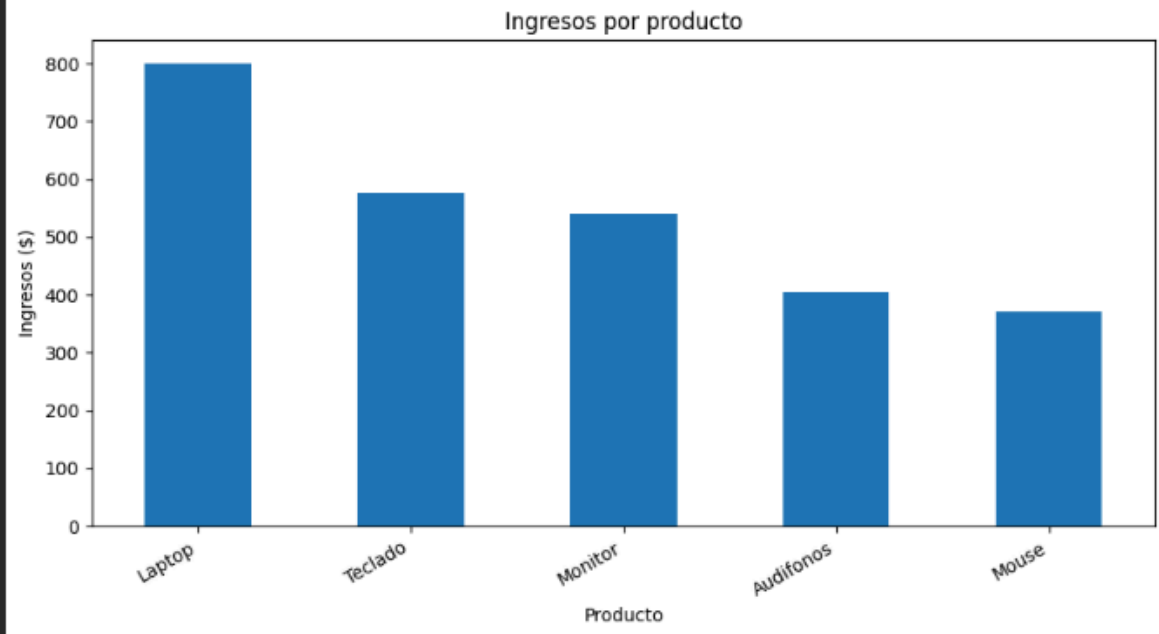


Gráfico guardado como ingresos.png.

Archivos generados:

ventas.csv: creado correctamente

log.txt: creado correctamente

ingresos.png: creado correctamente

README.md

1. Definición del catálogo
2. Registro de precios y stock
3. Creación de las listas de ventas
4. Creación de funciones
5. Registro de ventas
6. Implementación del descuento
7. Control de productos inexistentes
8. Implementación de Pandas
9. Generación del archivo CSV
10. Lectura del archivo CSV
11. Implementación de NumPy
12. Control de división por cero
13. Implementación de Matplotlib
14. Exportación de la gráfica
15. Implementación del Reto A
16. Implementación del Reto B
17. Implementación del Reto C
18. Implementación del Reto D
19. Creación del archivo de registro
20. Manejo de errores
21. Implementación de ciclos
22. Implementación del control de flujo
23. Realización de pruebas
24. Resultado final